
How-To's



- Threat Modeling Tool «Intro by MS»
 - <https://learn.microsoft.com/de-de/azure/security/develop/threat-modeling-tool>
- Funktionen des TMT
 - <https://learn.microsoft.com/de-de/azure/security/develop/threat-modeling-tool-feature-overview>
- Erste Schritte mit dem TMT
 - <https://learn.microsoft.com/de-de/azure/security/develop/threat-modeling-tool-getting-started>
- Threat Modeling Security Fundamental
 - <https://learn.microsoft.com/en-us/training/paths/tm-threat-modeling-fundamentals/>

Mitigation - How to / Process



- Typically, a **technical project manager takes ownership** of the threat model and schedules the analysis session
- The **architect, developer, and tester** should be present, and each threat should be **discussed**
- Some threats will be determined to have little impact on security objectives, while others need to be **mitigated**



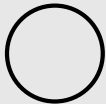
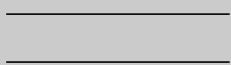
Mitigation is the Point of Threat Modeling

- Mitigation
 - → to address or lessen a problem
- Protect users
- Design secure software

Threat Properties			
ID: 1	Diagram: Diagram 1	Status: Not Started	Last Modified: Generated
Title: Potential Data Repudiation by Applications Running on a non Microsoft OS			
Category: Repudiation			
Description: Applications Running on a non Microsoft OS claims that it did not receive data from a source outside the trust boundary. Consider using logging or auditing to record the source, time, and summary of the received data.			
Justification:			
Interaction: HTTPS			
Priority: High			
Threat Properties Notes - no entries			

Cheat Sheet: Different Threats Affect Each Type



Element	S	T	R	I	D	E
External Entity 	x		x			
Process 	x	x	x	x	x	x
Data Store 		x	"?"	x	x	
Data Flow 		x		x	x	

Mitigate

- Address each threat (one by one)
- **4 ways** to address threats
 1. **Redesign** to eliminate
 2. Apply **standard mitigations**
 - What has worked for similar software packages?
 3. Invent **new mitigations** (riskier)
 4. **Accept vulnerability** in design
 - SDL rules about what you can accept (Bug Bar)



Mitigation Techniques

Threat	Steps to Mitigate	Techniques
Spoofting	1. Appropriate authentication 2. Protect secret data 3. Don't store secrets	Authentication: ▪ Cookie authentication ▪ Kerberos authentication ▪ PKI systems such as SSL/TLS and certificates To authenticate code or data: ▪ Digital signatures
Tampering	1. Appropriate authorization 2. Hashes 3. MACs 4. Digital signatures 5. Tamper resistant protocols	▪ ACLs ▪ Digital signatures



Mitigation Techniques

Repudiation	1. Digital signatures 2. Timestamps 3. Audit trails	▪ Secure logging and auditing
Information Disclosure	1. Authorization 2. Privacy-enhancing protocols 3. Encryption 4. Protect secrets 5. Don't store secrets	▪ Access Control lists (ACL)



Mitigation Techniques

Denial of Service	1. Appropriate authentication 2. Appropriate authorization 3. Filtering 4. Throttling 5. Quality of service	▪ ACLs ▪ Filtering ▪ Quotas
Elevation of Privilege	1. Run with least privilege	▪ ACLs ▪ Group or role membership ▪ Privilege ownership ▪ Input validation

Inventing Mitigations Is Hard: Be careful!



- Mitigations are an **area of expertise**, such as networking, databases, or cryptography
- Amateurs make **mistakes**, but so do pros
- Mitigation failures **may appear to work...** until they don't...
- When you need to invent mitigations, get expert support

Validate and Review



- Any **residual risks** should be escalated for **management review** to determine if an unmitigated threat is **acceptable to the project as a whole**.
- The **entire team benefits** by engaging in the threat modeling. The involvement leads to raised awareness of the implications of design and implementation decisions.
- Additional benefits may include higher overall **code quality** as well as greater **management visibility of security controls**.

Validating Threat Models

IMPORTANT



- **Validate the whole threat model**
 - Are threats enumerated?
 - **Minimum:** STRIDE per element that **touches a trust boundary**
 - Has **Test / QA reviewed** the model?
 - Tester approach often finds issues with threat model or details
 - Is **each threat** addressed?
 - Are mitigations done right?
- Did you check these **before Final Security Review**?
 - Shipping will be more predictable

Validate Quality of Threats and Mitigations



- Threats - do they:
 - Describe the attack
 - Describe the context
 - Describe the impact
- Mitigations
 - Associate with a threat
 - Describe the mitigations
 - File a bug
 - Fuzzing is a test tactic, not a mitigation

Validate Captured Information



- **Dependencies**
 - What other code are you using?
 - What security functions are in that other code?
 - Are you checking the data provided by external services?
 - Are you sure?



Naming @Threat Model you build!

- Make your flows be meaningful
 - Bad: "HTTP", "Request"
 - Good: "Blog Content", "Scrape Requests"
- Name your trust boundaries
 - Machine?
 - Corporate?
 - Network?
- Processes & external interactors
 - Good: "<company>.com"
 - Bad: "server"
- Ultimate test: outsiders understand your system without other info than the data flow diagram

More Tips for Threat Modeling

Be honest with the process

- Make sure the model represents reality (or what you really believe reality will be)
- Consider all types of threats

As with all modeling, use appropriate complexity

- Overly-simplified?
 - Departs from reality
 - You get exactly what you put into it – no new knowledge
- Overly-complicated?
 - Too much to analyze
 - “Check it off the list” syndrome
- Test your model
 - Think of a specific security concern, then try to see where it fits in your threat model



Risk Rating



DREAD – How to rate the RISK

A methodology for risk rating

Each threat is graded in this categories:

- Damage - how bad would an attack be?
- Reproducibility - how easy is it to reproduce the attack?
- Exploitability - how much work is it to launch the attack?
- Affected users - how many people will be impacted?
- Discoverability - how easy is it to discover the threat?

Some examples



Rating	High (3)	Medium (2)	Low (1)
Damage potential	The attacker can undermine the security system; get full trust authorization; run as administrator; upload content.	Leaking sensitive information	Leaking trivial information
Reproducibility	The attack can be reproduced every time and does not require a timing window.	The attack can be reproduced, but only with a timing window and a particular race situation.	The attack is very difficult to reproduce, even with knowledge of the security hole.

Some examples



Rating	High (3)	Medium (2)	Low (1)
Exploitability	A novice programmer could make the attack in a short time.	A skilled programmer could make the attack, then repeat the steps.	The attack requires an extremely skilled person and in-depth knowledge every time to exploit.
Affected users	All users, default configuration, key customers	Some users, non-default configuration	Very small percentage of users, obscure feature; affects anonymous users

Some examples



Rating	High (3)	Medium (2)	Low (1)
Discoverability	Published information explains the attack. The vulnerability is found in the most commonly used feature and is very noticeable.	The vulnerability is in a seldom-used part of the product, and only a few users should come across it. It would take some thinking to see malicious use.	The bug is obscure, and it is unlikely that users will work out damage potential.

DREAD - Calculation

Risk rating	Result
High	11 - 15
Medium	8 - 11
Low	5 - 7

Threat	D	R	E	A	D	Total	Rating
Attacker obtains authentication credentials by monitoring the network.	3	3	2	2	2	12	High
SQL commands injected into application.	3	3	3	3	2	14	High

Some security experts feel that including the "Discoverability" element as the last D rewards **Security through obscurity**, so some organizations have either moved to a **DREAD-D** "DREAD minus D" scale (which ignores Discoverability) or always assume that Discoverability is at its maximum rating.

Threat Modeling - OWASP



- OWASP Threat List
 - Authentication
 - Authorization
 - Configuration Management
 - Data Protection in Storage and Transit
 - Data/Parameter Validation
 - Error Handling and Exception Management
 - User and Session Management
 - Auditing and Logging

Threat Modeling - OWASP



OWASP Risk Rating

- 4 x 4 factors are evaluated (1-9 points)
- Overall risk is computed to medium - high - critical

Threat Modeling - OWASP



OWASP Risk Rating

- Threat Agent: Skill level, Motive, Opportunity, Size
- Vulnerability: Ease of discovery, Ease of exploit, Awareness, Intrusion detection
- Technical Impact: Loss of confidentiality, Loss of integrity, Loss of availability, Loss of accountability
- Business Impact: Financial damage, Reputation damage, Non-compliance, Privacy violation

Common Vulnerability Scoring System



- The Common Vulnerability Scoring System (CVSS) provides a way to capture the principal characteristics of a vulnerability, and produce a numerical score reflecting its severity, as well as a textual representation of that score.
-
- CVSS affords three important benefits.
 - it provides standardized vulnerability scores
 - it provides an open framework
 - enables prioritized risk
- <https://www.first.org/cvss/specification-document>

And some other ways...



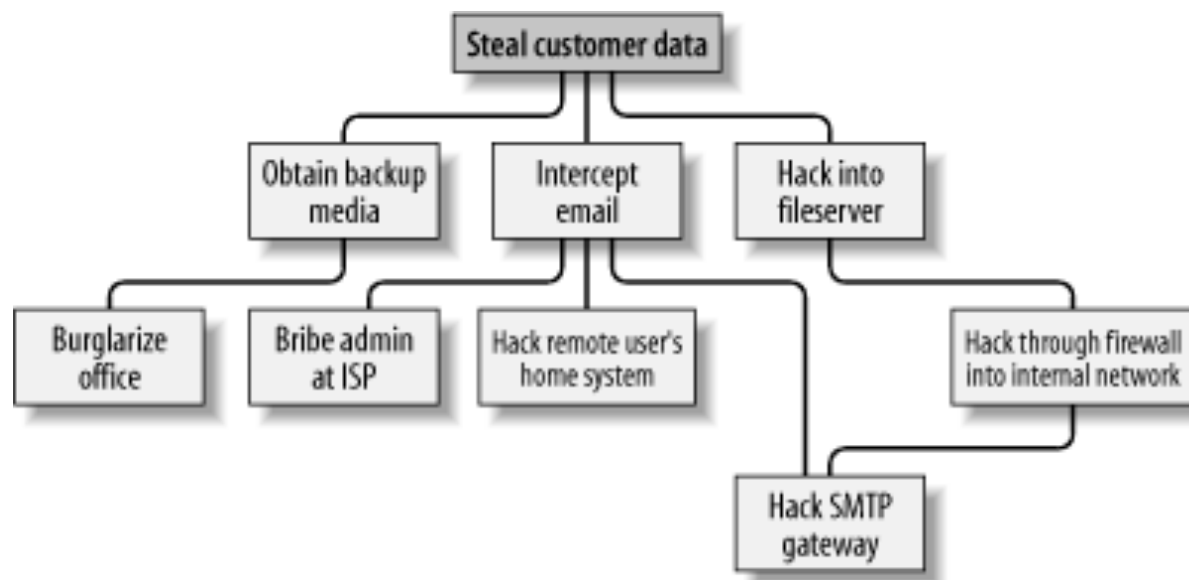
- Pasta
 - Security Cards
 - Persona non grata
 - ...
-
- https://insights.sei.cmu.edu/sei_blog/2018/12/threat-modeling-12-available-methods.html



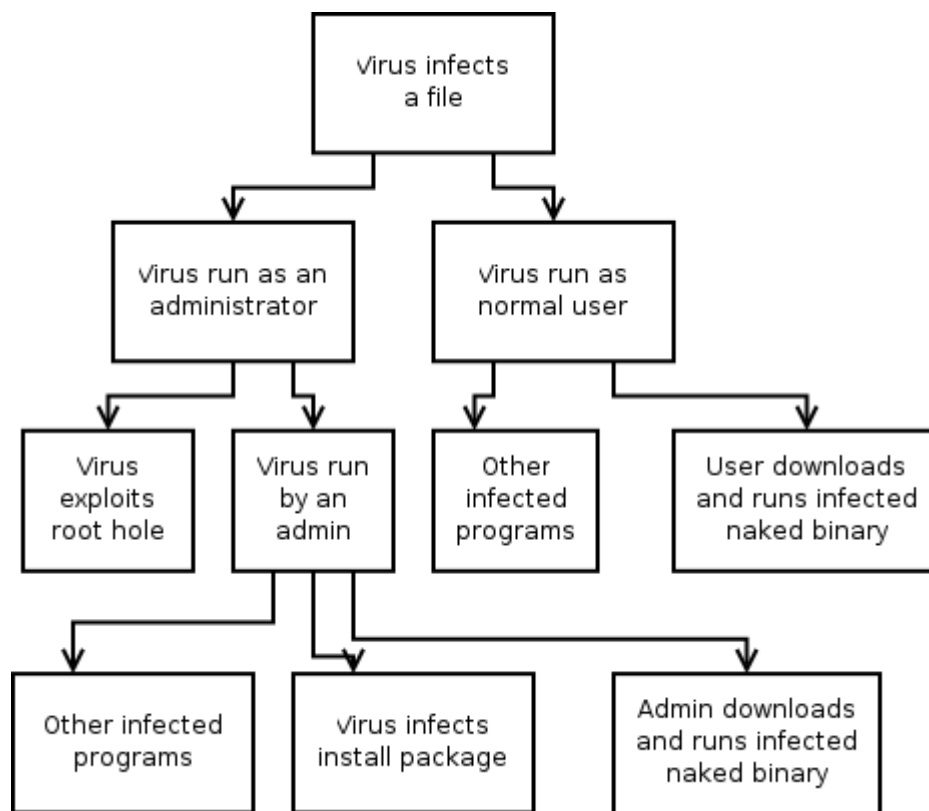
Attack Tree Example

By P. Tivolata

Attack Trees „steal customer data“



Attack Trees „Virus infection“



Attack Trees - Types of Nodes

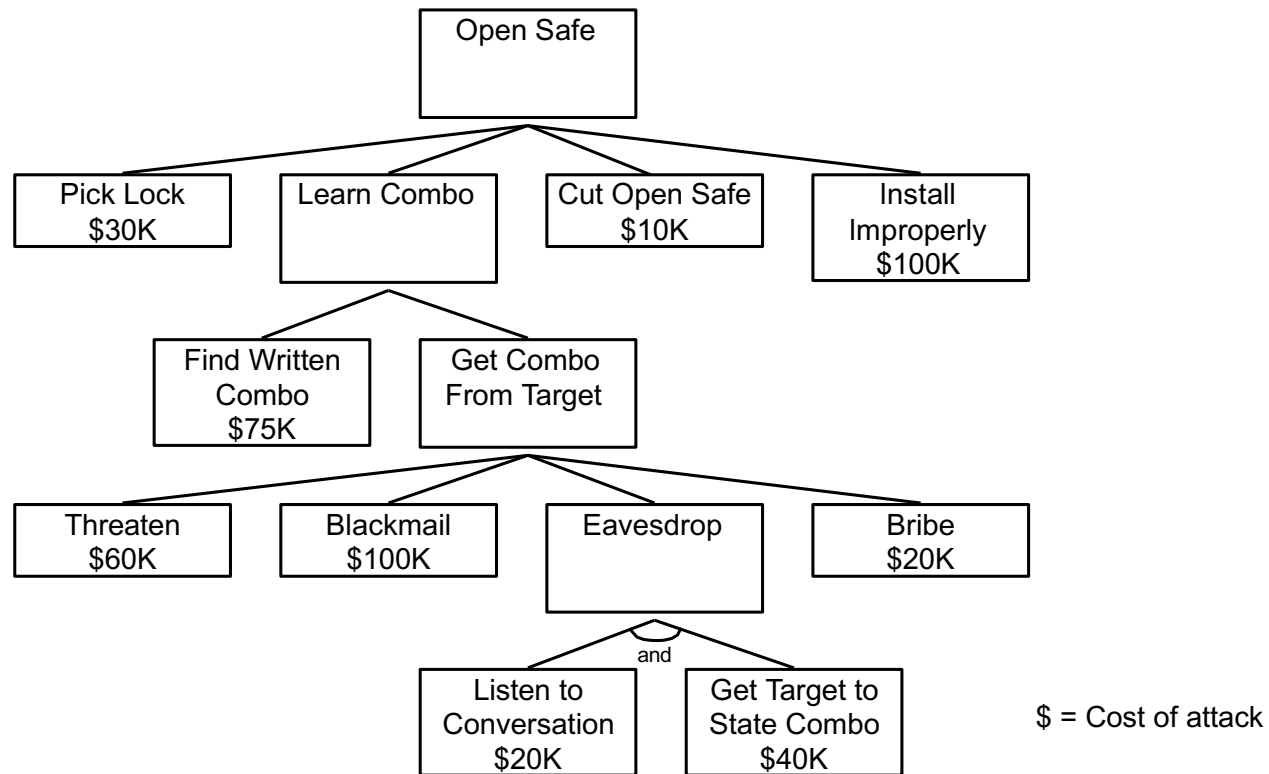


- "OR" nodes represent different ways to achieving the same goal.
 - For example, to break into a house you can either pick the door lock **OR** break a window.
- "AND" nodes represent different steps in achieving a goal
 - For example, to enter through a window you need to break the window **AND** climb through the opening.

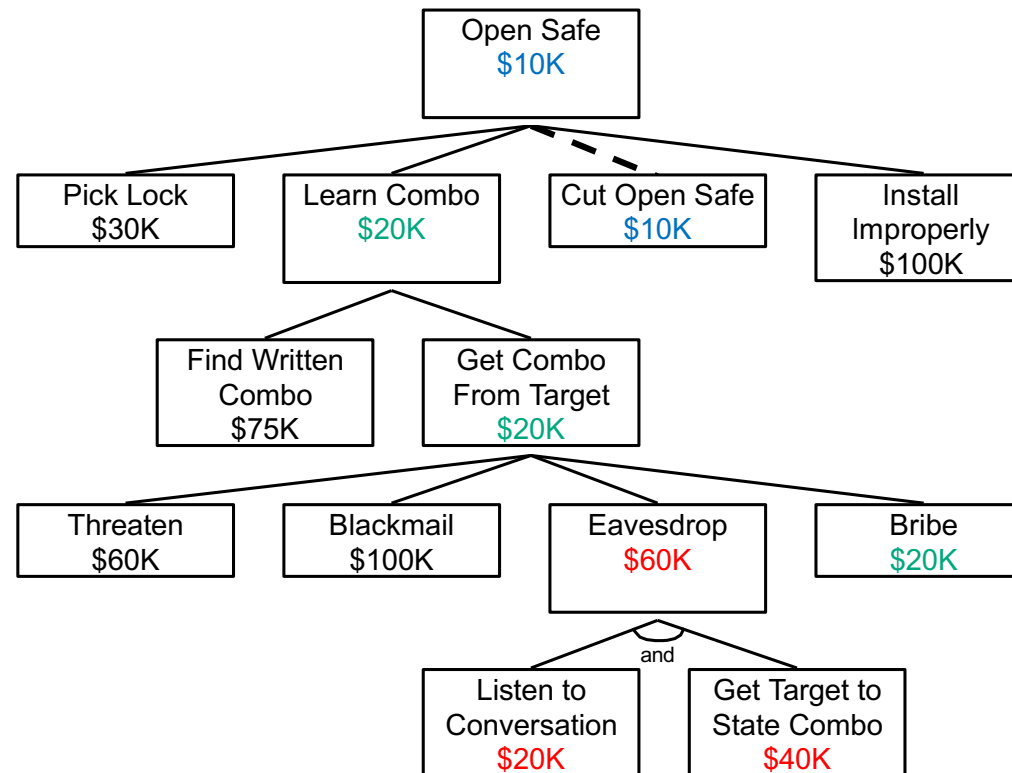
Attack Trees – Node Values

- Once a tree is created, **different values** can be assigned to the leaf nodes
 - Boolean: possible vs. Impossible
 - Cost (money or resources) to attack / Cost (money or resources) to defend
 - Time to achieve / Time to repulse
 - Probability of success of a given attack.
 - Likelihood that an attacker will try a given attack
- Values of the other nodes can be calculated:
 - OR nodes have the value of their cheapest child
 - AND nodes have the value of the sum of their children

Attack Trees - Node Values



Attack Trees - Node Values

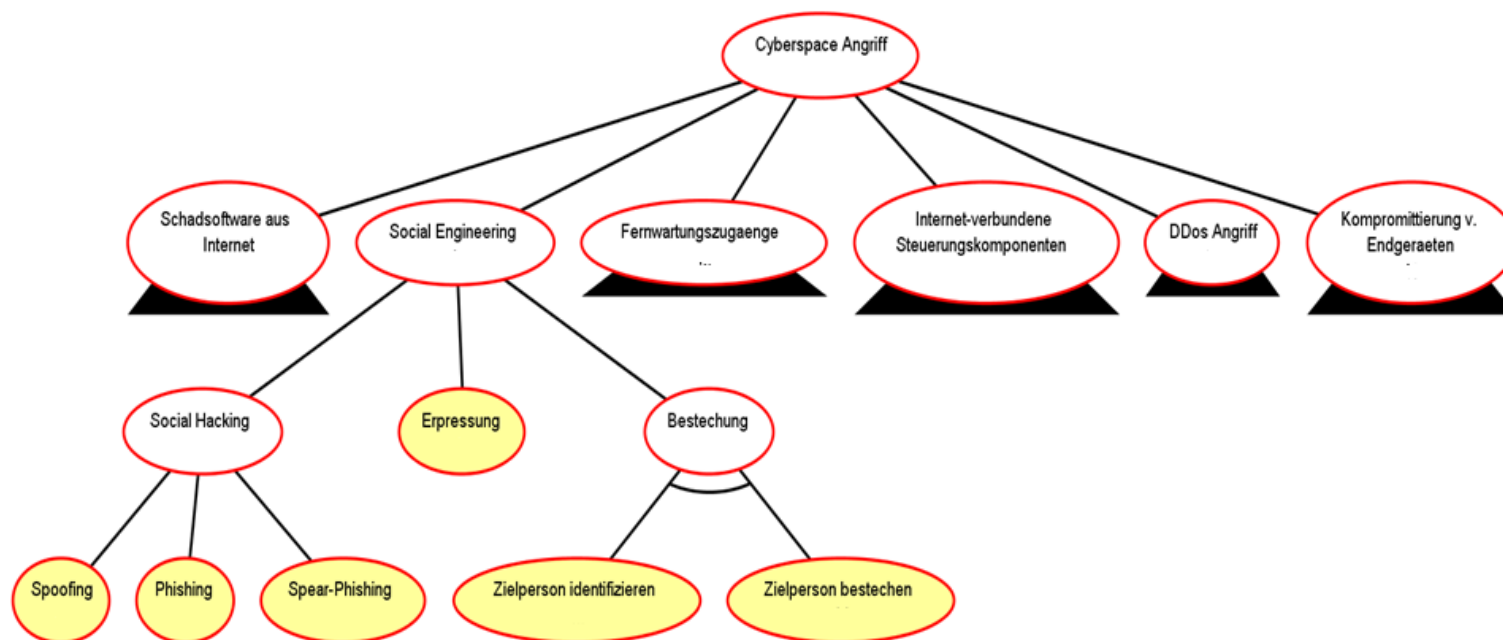


\$ = Cost of attack

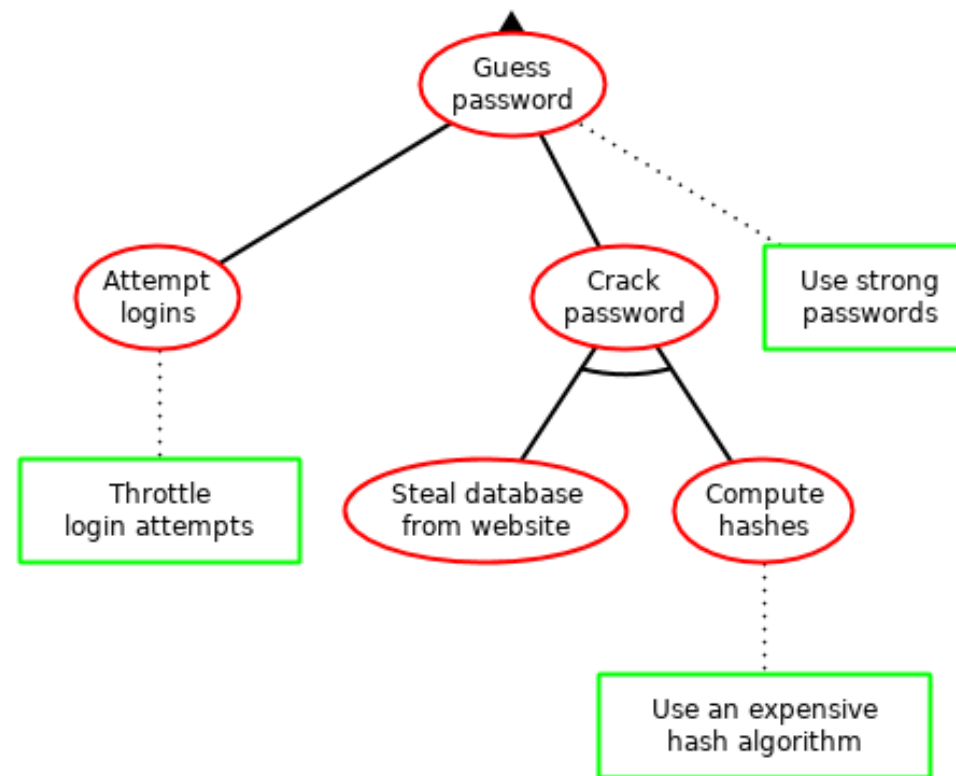


Attack Tree Example

Example - Industrial Control System



Countermeasures



<https://bbc.re/blog/security-password-authentication/>